

Architecture & ER Diagram

1. System Overview

The platform orchestrates Wazuh, Suricata, Zeek, Falco, TheHive, n8n, and Keycloak — all unmodified open-source components behind a custom FastAPI backend that handles integration, AI enrichment, and multi-tenant management. A Next.js frontend consumes the backend's unified REST/WebSocket API; Traefik terminates TLS and routes three public subdomains.

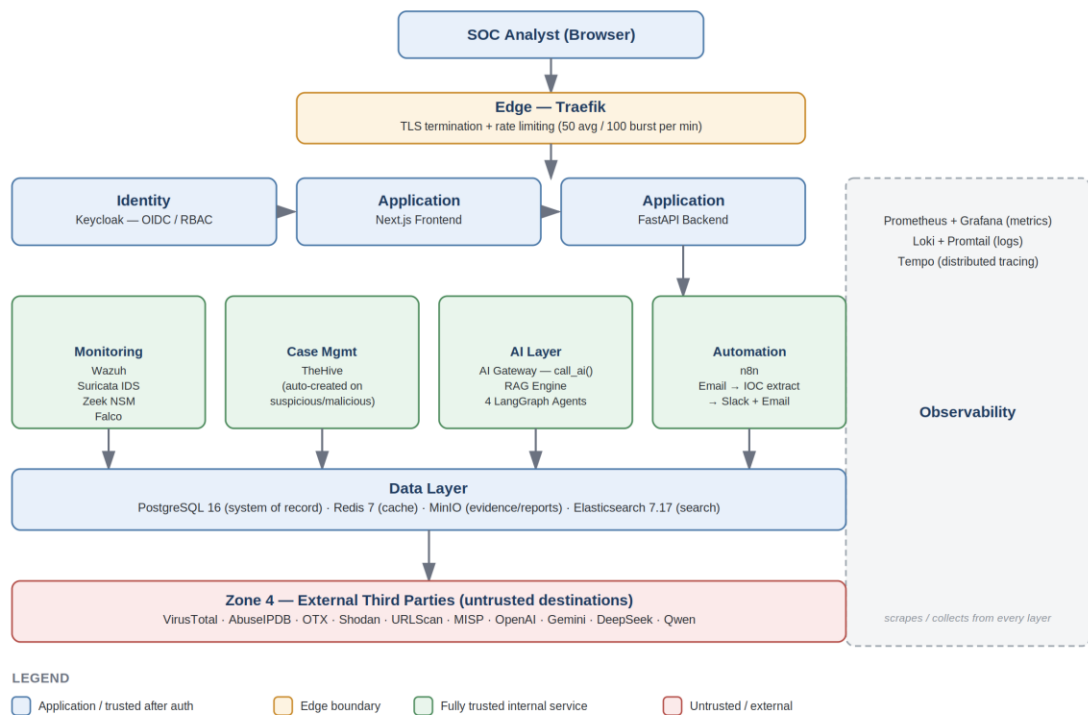


Figure 1.1 — High-level system architecture and data flow. Arrows show the primary request path.

Figure 1.1 — High-level system architecture and data flow. Arrows show the primary request path.

Layer Breakdown

Layer	Component	Responsibility
Edge	Traefik	TLS termination (Let's Encrypt), subdomain routing, rate limiting (50 avg / 100 burst per min)
Identity	Keycloak	OAuth2/OIDC, RS256 JWT issuance, RBAC via realm roles
Application	Next.js / FastAPI	SOC UI (10 pages); backend aggregates every tool's API, AI orchestration, tenant mgmt, REST + WebSocket
Monitoring	Wazuh, Suricata, Zeek, Falco	Host SIEM/XDR, network IDS, network security monitor, runtime/syscall security — all log-ingested by the backend
Case Mgmt	TheHive	Auto-creates a case whenever a threat check returns suspicious/malicious

Layer	Component	Responsibility
Threat Intel	VirusTotal, AbuseIPDB, OTX, Shodan, URLScan, MISP	Normalized into the internal Indicator model
AI	AI Gateway, RAG Engine, LangGraph Agents	Single call_ai() entry point across 5 providers; Gemini-embedded RAG; 4 autonomous agents
Automation	n8n	Email-triggered enrichment pipeline → Slack + email notification
Data	PostgreSQL 16, Redis 7, MinIO, Elasticsearch 7.17	System of record, fail-soft cache, evidence/report storage, unified search
Observability	Prometheus/Grafana, Loki/Promtail, Tempo	Metrics + dashboards, log aggregation, distributed tracing

Network & Deployment Topology

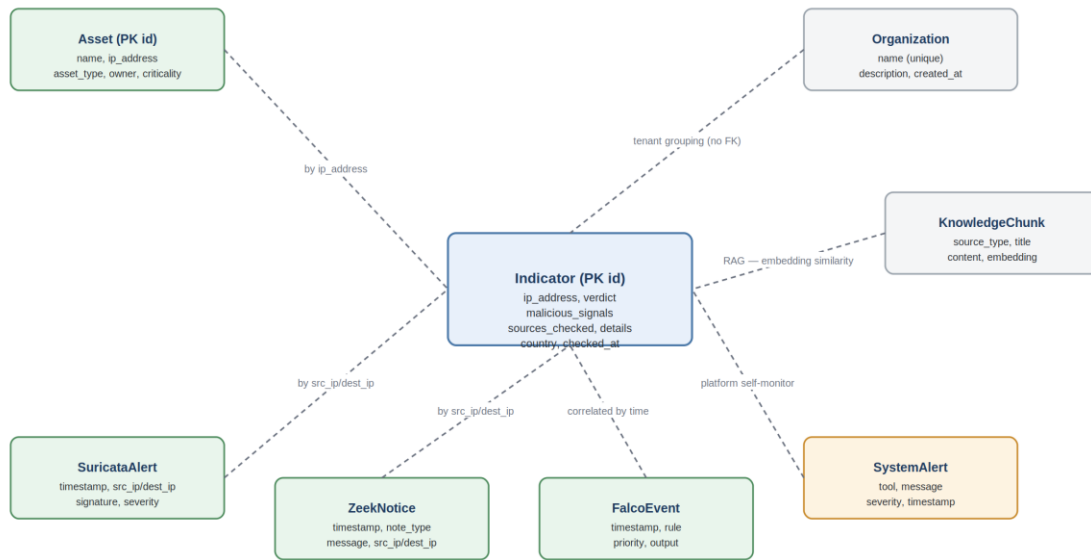
- Single Docker Compose stack (sop-network, bridge), ~22 containers on one Contabo VPS (V45: ~4 vCPU / 8GB RAM, Ubuntu 22.04, 4GB swap via cloud-init).
- Public entry points behind Traefik + Let's Encrypt: app.169-58-221-49.nip.io (frontend), api.169-58-221-49.nip.io (backend, rate-limited), n8n.169-58-221-49.nip.io.
- Everything else Postgres, Redis, MinIO, Keycloak, Elasticsearch, TheHive, Wazuh, Suricata, Zeek, Falco, observability stack — is internal-only, reached via container DNS.
- Suricata/Zeek/Falco run with elevated host privileges (NET_ADMIN/NET_RAW, or full privileged + pid:host for Falco) for live traffic/syscall capture.

Key Architectural Decisions

- No custom SIEM/IDS/workflow engine. Wazuh, Suricata, Zeek, Falco, n8n run as-is; the backend only orchestrates and normalizes.
- Dual Wazuh access: Manager REST API for health/status/agents only; actual alerts pulled from the Wazuh Indexer (OpenSearch) directly, since the Manager API has no /alerts endpoint.
- Fail-soft integrations: Redis caching, Elasticsearch indexing, and WebSocket broadcast are wrapped so a failure in any never breaks the core threat-check flow.
- Single normalized threat model: the Indicator table is what all four TI sources normalize into, driving both case auto-creation and AI enrichment.
- Provider-agnostic AI Gateway: every AI feature routes through one call_ai(prompt, provider, feature) function provider choice is a request-time parameter, not hardcoded.

2. Entity-Relationship Model

The data model is intentionally simple: no formal foreign keys between operational tables. Records are correlated at query time by IP address and timestamp, keeping ingestion fast and tolerant of partial/out-of-order data from independent log sources that were never designed to share a schema.



No formal foreign keys — all correlation happens at query time by IP address / timestamp / embedding similarity (dashed lines).
Figure 2.1 — Core PostgreSQL entities and correlation strategy.

Figure 2.1 — Core PostgreSQL entities and correlation strategy. No formal foreign keys; all correlation happens at query time by IP/timestamp (dashed lines).

Entity	Purpose	Key Fields
Indicator	Normalized result of a unified threat-intel check on an IP	ip_address, verdict, malicious_signals, sources_checked (JSON), details (JSON), country, checked_at
Asset	Inventoried org assets	name, ip_address, asset_type, owner, criticality, status
Organization	Tenant records	name (unique), description
SuricataAlert / ZeekNotice / FalcoEvent	Deduplicated ingested security events	timestamp, src_ip/dest_ip, signature/rule, severity/priority, raw_event (JSON)
SystemAlert	Platform self-monitoring (tool silence, real Wazuh alerts)	tool, message, severity
KnowledgeChunk	RAG knowledge base — Sigma rules, MITRE ATT&CK, CVEs, playbooks	source_type, title, content, embedding (JSON)

Correlation Strategy

- Indicator ↔ Asset: correlated by IP address when cross-referencing a finding against inventory.
- SuricataAlert / ZeekNotice ↔ Indicator: correlated by src_ip/dest_ip in /alerts/unified and /search.
- KnowledgeChunk ↔ Indicator: pulled at query time by the RAG engine via embedding similarity — not a stored relationship.
- All operational tables are also mirrored into Elasticsearch on write, independent of the Postgres correlation above.

Reflects the deployed implementation as of the current codebase.